Cahier des charges

Anais Azouaoui et Caroline Zouloumian

Fevrier 2026

Contents

1 Introduction **2**

2 Objectifs **2**

3 Description du système **2**
3.1 Module d'interprétation sémantique (LLM / VLM) 3
3.2 Module de génération de scène Genesis 3
3.3 Boucle de validation et correction 4
3.4 Interface utilisateur 4
3.5 Architecture globale 4

4 Contraintes **5**

5 Évaluation et critère d'acceptation **6**

6 Planning **6**
6.1 Deadlines générales du projet AI2D 6
6.2 Notre organisation 6

1 Introduction

L'apprentissage en robotique nécessite généralement un entraînement en conditions réelles chronophage et mobilisant du matériel spécialisé onéreux. La simulation constitue ainsi une alternative privilégiée. Toutefois, la création manuelle de scène virtuelles reste elle même longue et peu scalable et limite la diversité et fréquence des expérimentations.

Ce projet, réalisé dans le cadre de l'UE PAI2D du Master 1 AI2D à Sorbonne Université, et encadré par le chercheur postdoctoral Emiland Garrabe et le Professeur Stéphane Doncieux de l'Institut des Systèmes Intelligents et de Robotique (ISIR), vise à automatiser la génération de scène de simulation robotique grâce aux modèles fondations. Ce dernier a pour ambition de réduire l'écart entre la simulation et la réalité.

Notre système sera capable de produire automatiquement des environnements simulable réalistes à partir de prompt textuels ou d'images, et prendra la forme d'un logiciel intermédiaire entre un Modèle Vision Language (VLM) et le moteur de simulation Genesis, chargé de traduire des descriptions sémantique ou visuelles en configurations exploitables.

2 Objectifs

L'objectif de ce projet est de développer un logiciel capable de générer automatiquement des scènes 3D simulables à partir d'une entrée textuelle (prompt) et/ou d'une image de référence.

Le système doit agir comme une couche intermédiaire entre des modèles fondations (LLM et VLM) et le moteur de simulation Genesis, en traduisant une description sémantique ou visuelle en une configuration exploitable par l'API Python du simulateur.

Plus précisément, le logiciel devra :

- Produire un fichier de configuration au format JSON contenant les informations nécessaires à la génération d'une scène 3D valide sous Genesis, à partir d'un prompt ou d'une image de référence analysés par les modèles fournis via Ollama (Llama 3.1 pour le texte et Llama 3.2 pour la vision).
- Instancier, simuler et tester automatiquement la scène générée dans Genesis
- Garantir la cohérence physique et la fidélité de la scène générée :
  - objets en lévitation, interpénétration, collision, etc.
  - respecter les relations spatiales et dimensions relatives
  - rester fidèle à la description en entrée
- Permettre potentiellement une extension future du système, notamment :
  - l'ajout ou la modification d'objets sur commande,
  - l'amélioration progressive de la précision géométrique,

Une phase de validation sur robot réel sera également possible sous réserve de temps disponible.

3 Description du système

Notre logiciel traduira des entrées en une scène de simulation robotique. Celles ci pourront être : des prompts textuels, qui consistent en des indications sur la scène à configurer ("Crée une scène pour l'apprentissage de saisie de mug", ou encore "Mets un crayon à droite du mug sur la table" par exemple) ou des images de référence (photographie de la scène à reproduire). En entrée pourront aussi figurer des mesures physiques fournies par l'utilisateur. La sortie sera une scène générée sur Genesis qui correspondra aux instructions données.

Une bibliothèque d'objet URDF (Unified Robot Description Format) prédéfinie sera fournie par l'ISIR. Ce projet ne nécessite donc pas de générer les objets en eux-mêmes.

Le logiciel reposera donc sur une architecture modulaire en plusieurs blocs.

3.1 Module d'interprétation sémantique (LLM / VLM)

Ce premier module sera responsable de l'analyse de l'entrée de l'utilisateur :

Cas 1 : Prompt textuel. Dans le cas d'une entrée textuelle (exp : "Place un mug et un livre sur une table") le modèle Llama 3.1 sera chargé de :

- Identifier les objets présents dans la scène (mug,table,livre). Les objets seront limités à ceux présents dans la bibliothèque URDF.
- Extraire leurs relations spatiales (sur, sous, derrière, etc.)
- En déduire leur positions et dimensions relatives
- Générer une représentation structurée au format JSON, par exemple :

```
{
    "object_type": "mug",
    "x": 0.0,
    "y": 0.0,
    "z": 0.0,
    "quaternions": [],
}
```

Cas 2: Image de référence. Dans le cas d'une image, nous exploiterons le modèle Llama 3.2-Vision dans le but de :

- Détecter les objets principaux de la scène.
- Déterminer la pertinence de ceux ci au problème (lorsque l'objectif est d'entraîner un bras robotique à agripper une tasse, la présente d'autres objets sur la table n'est pas toujours pertinente)
- Estimer leurs proportions relatives, leur position approximative et leurs relations spatiales.
- Générer une représentation structurée intermédiaire au format JSON

Dans ce mode, l'objectif est de générer une scène 3D la plus fidèle possible à l'image de référence. Pour se faire, un socle d'itération et de validation, détaillé dans une prochaine partie, sera mis en place.

Le module LLM/VLM produit ainsi un fichier JSON contenant le type des objets, leurs dimensions, leurs coordonnées 3D normalisées, leurs orientations (quaternions) et leurs relations structurelles relatives.

Cas 3: Mesures réelles. Dans ce mode, des mesures physique réelles de la scène à reproduire viendront complémenter un prompt ou une image. Ce module est utile dans le cas du besoin de placer un objet dans une position très précise.

3.2 Module de génération de scène Genesis

Ce module convertira une configuration JSON en instructions exploitables par l'API Genesis.

Il sera responsable de :

- Créer une scène vide dans l'environnement de simulation et charger les objets 3D identifiés en les associant aux assets de la bibliotheque URDF (Unified Robot Description Format) de l'ISIR

- Instancier chaque objet et appliquer positions et orientations.
- Vérifier la bonne initialisation des entités (collisions, autrescontraintes éventuelle) et initier la simulation.

3.3 Boucle de validation et correction

La boucle de validation itérative permet d'éviter les comportements irréalistes et de vérifier le respect des lois de la physique. Les objets devront être décollés du sol, par exemple, pour pouvoir être saisis.

Dans le cas d'une entrée visuelle, cette boucle pourra effectuer les vérifications de validité physique et comparer le résultat à l'image de référence.

Dans le cas d'une entrée textuelle, la partie validation via VLM aura principalement pour rôle de vérifier les inconsistances physique (une mauvaise position d'un objet, collision, interpénétration) et d'aider à la correction.

La boucle consiste à :

- Comparer la scène générée obtenue avec l'image de référence et identifier les divergences majeures (objets manquants, mauvaise disposition etc.), via VLM, et/ou vérifier le respect des lois physiques et la cohérence de la scène.
- Ajuster les paramètres de la scène en conséquence.
- Itérer jusqu'à atteindre un seuil de fidélité à définir.

Ce mécanisme itératif a pour ambition d'améliorer progressivement la fidélité structurelle entre l'image/le prompt et la scène simulée.

3.4 Interface utilisateur

Le système comprendra une interface minimale permettant de soumettre un prompt ou une image, ainsi qu'une visualisation de la scène générée.

3.5 Architecture globale

La pipeline générale du système est le suivant :

- Entrée utilisateur (texte ou image)
- Interprétation LLM/VLM
- Génération d'un JSON structuré
- Conversion vers API Genesis
- Génération de la scène simulable
- Boucle de validation itérative
- Export

Cette pipeline est également représentée par un flowchart dans la *Figure 1*.

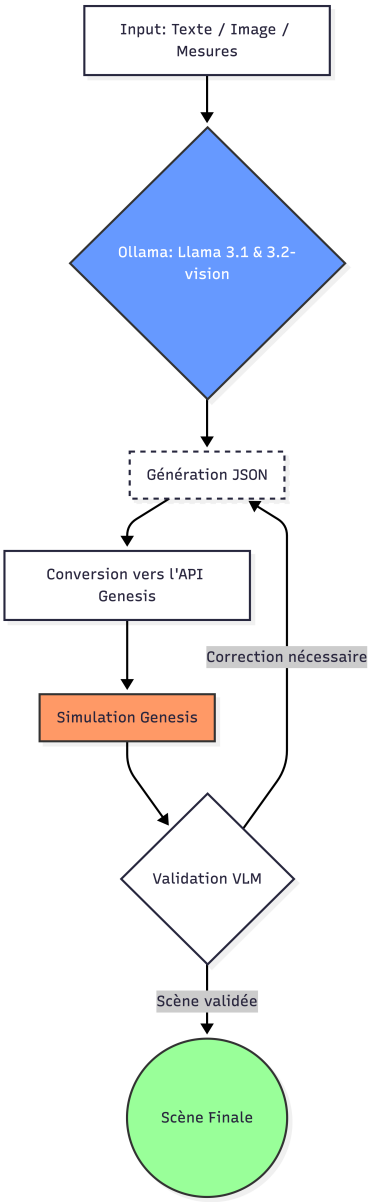

Figure 1: Flowchart représentant la pipeline générale du système.

4 Contraintes

Contraintes logicielles: Nous utilisons les modèles llama 3.1 et 3.2-vision avec Ollama pour l'automatisation de la génération de scène. Genesis (simulateur python) nous permet de générer les instances de scènes de simulation. Les objets pour la simulation viendront d'une bibliothèque d'objets URDF (Unified Robot Description Format) de l'ISIR.

Contraintes matérielles: La simulation Genesis et l'utilisation de LLM nécessitent des ressources GPU conséquentes. Le processus de simulation est donc lent sur certaines machines. Le bras Emika Franka de l'ISIR sera nécessaire dans le cas d'une validation sur robot réel.

5 Évaluation et critère d'acceptation

La phase de test comprendra la génération et reproduction de scènes. On donnera au logiciel une scène générée par le passé pour vérifier à quelle précision celui ci pourra la reproduire. La génération devra également relativement rapide. On regardera plusieurs critères:

Précision: un certain écart minimal entre la position réelle demandée et la simulation. La précision est surtout intéressante lorsque la simulation est basée sur des mesures réelles.

Validité/vraisemblance: la validité physique de la scène.

Taux de succès en 'one-shot': la proportion de scènes valide (sans collision, objets correctement placés) sans intervention humaine.

Transférabilité, reproductibilité des résultats dans le monde réel (sim-to-real): succès d'une tâche sur le bras Franka Emika, basée sur la scène générée.

6 Planning

6.1 Deadlines générales du projet AI2D

- **Fin du premier mois du projet:** rendu du cahier des charges.
- **18 mai:** rendu du rapport.
- **21-22 mai:** soutenance.

6.2 Notre organisation

- **26 février:** rendu du cahier des charges.
- **8 mars:** fonctionnement du simulateur à partir de prompts.
- **29 mars:** fonctionnement du simulateur à partir de prompts.
- **1er avril:** fin du développement logiciel.
- **12 avril:** fin des expérimentations.
- **1er mai:** première version du rendu.
- **5 mai:** version finale du rendu.
- **18 mai:** rendu du rapport.
- **21-22 mai:** soutenance.

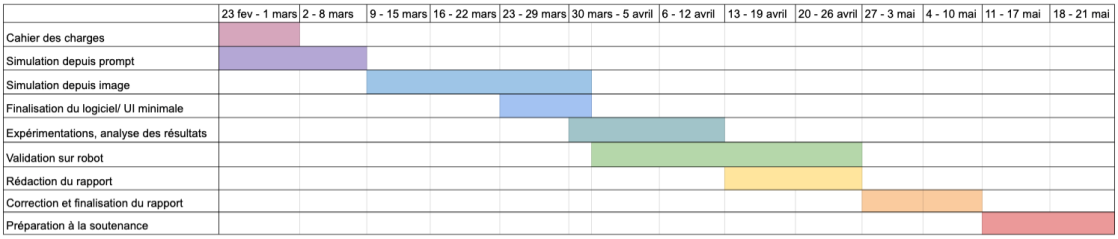

Figure 2: Diagramme de Gantt représentant les étapes du projet dans les semaines du semestre